Report I - SF2957 Statistical Machine Learning

Vincent Hallberg [redacted] Ville Wassberg [redacted]
Klara Zimmerman [redacted] Noel Yonatan Efrem [redacted]

November, 2024

Classification of handwritten digits

Objectives

The goal of this project is to classify handwritten digits from the MNIST database by implementing and comparing three different classification methods: Support Vector Machines (SVM) using projected stochastic subgradient descent, Gaussian processes (GPs), and convolutional neural networks (CNNs).

We first implement the SVM, which aims at maximizing the distance from data points in each class to the margin that separates the classes. This is done by minimizing a convex multi-class hinge loss function. The classifier is iteratively updated using stochastic subgradients, and projected onto a given set. The performance is evaluated using different training-set sizes and different learning rates.

We then evaluate a GP multi-class classifier and a GP regression-based classifier on the MNIST dataset. Both use kernel functions to determine the covariance structures. The GP classifier uses a one-versus-rest approach, where multiple binary classifiers are fitted to predict class probabilities. The GP regression-based classifier uses a one-hot representation of the labels and predicts continuous outputs, where the final class is determined by the largest predicted value. Both models are trained using various kernels and training set sizes to assess their performance.

Finally, we implement a CNN, a deep learning model designed to efficiently recognize patterns in image data. The performance of this method is evaluated using different training set sizes and different layer architectures.

Mathematical Background

Convex optimization and the Projected Subgradient Method

We define a function $G : R^d \to R$ as **convex** if for any $[x, y] \in R^d$, and $t \in [0, 1]$, we have

$$G(tx + (1-t)y) \leq tG(x) + (1-t)G(y). \tag{1}$$

Such functions are differentiable almost everywhere, and they have a **subgradient set**, $\delta_G(\theta)$, everywhere. This set is defined as

$$\delta_G(\theta) := \{g \in R^d : \forall \psi \in R^d : G(\theta) + g^T(\psi - \theta) \leq G(\psi)\}. \tag{2}$$

The subgradient set thus provides a generalization of the gradient in non-differentiable points of a convex function.

If we want to minimize G over a compact convex set C, we can use the **Projected Subgradient Method**. The projection operator is defined by $proj_C(\theta) = argmin_{\psi \in C}|\psi - \theta|$. The algorithm in the projected setting is:

$$\begin{aligned} &\text{Initialize } \theta_1, \text{ and iterate:} \\ &\qquad \text{for } \theta_n, \text{ take } g_n \in \delta_G(\theta_n), \\ &\qquad\qquad \text{set } \epsilon_n \text{ as stepsize}, \\ &\qquad\qquad \tilde{\theta}_{n+1} = \theta_n - \epsilon_n g_n, \\ &\qquad\qquad \theta_{n+1} = proj_C(\tilde{\theta}_{n+1}). \end{aligned} \tag{3}$$

Gaussian Process classification In multi-class GP classification we have a prior on a random function for each class $f(x) \sim \mathcal{N}(m(x), k(x, x'))$, where $m(x)$ is the mean function and $k(x, x')$ is the covariance function. For the classifier this function is (automatically) transformed into class probabilities by the softmax function, providing a predictive distribution as

$$p(y \mid x) = \int_\Theta \prod_{c=1}^{C} \text{softmax}(f_\theta(x_{n+1}))_c^{\mathbb{I}\{y=a_c\}} p_{\Theta|X_1,Y_1,\ldots,X_n,Y_n}(\theta \mid x_1, y_1, \ldots, x_n, y_n)\, d\theta,$$

which is not approximated by a true Laplace approximation but rather simplifies the multi-class classification into several binary classifiers in the implementation in Scikit. Then the predicted class for an input x_i is expected to be the one maximising the predictive posterior $\hat{y} = \arg\max_c p(y = c|x_i)$.

In the regression case, the action space is encoded such that $\aleph = \{a_1, \ldots, a_C\}$ where a_c is a one-hot-vector with a 1 in the c^{th} component, e.g. for class 1, being 0 in our case, we have the unit vector e_1. Then the formal Bayes rule is found by taking the maxiser of the predictive probabilities $p_{Y|X}(a_c|x)$, $c = 1, \ldots, C$. Here we expect the posterior to be Gaussian, since both prior and likelihood are Gaussian. The regression predicts $f_c(x)$ for each class, which can be interpreted as scores of how certain it is a data point belongs to the class c. Then the classification is made by labeling as the class maximising this score, hence, $\hat{y}_i = \arg\max_c f_c(x_i)$.

Convolutional neural networks

A convolutional neural network uses one or more blocks of layers. Each layer consists of a convolutional layer, a detection layer, a pooling layer combined with a fully connected layer at the end.

Firstly the image is represented with a 3D tensor (or 2D tensor if the it has one channel) with a $n \times m$ matrix and c channels, which is a 3D tensor $n \times m \times c$. Let's assume that we work with only one channel (but the generalization can still be extended to 3D).

1. Convolution Operation: A convolutional layer applies a $k \times k$ kernel W to the input I with stride 1 and no padding. The resulting feature map H of size $(n-k+1) \times (m-k+1)$ is computed as:

$$H(i,j) = \sum_{k=0}^{K-1} \sum_{l=0}^{K-1} W(k,l) I(i+k, j+l) + b,$$

where b is the bias term, and $0 \leq i \leq n-k$, $0 \leq j \leq m-k$.

2. Activation Function: An activation function $g(x)$ is applied element wise to the feature map $H(i,j)$. For example, if g is the ReLU function, $g(x) = \max(x, 0)$, the detection layer is:

$$H^{\text{out}}(i,j) = g(H(i,j)).$$

3. Pooling Operation: A pooling layer is then applied to the detection layer H^{out}. Using a $p \times p$ max pooling operation with stride s, the pooling layer output P is computed as:

$$P(p,q) = \max\{H^{\text{out}}(sp, sq), H^{\text{out}}(sp, sq+1), H^{\text{out}}(sp+1, sq), H^{\text{out}}(sp+1, sq+1)\},$$

for $0 \leq p \leq \frac{n-k}{s} - 1$ and $0 \leq q \leq \frac{m-k}{s} - 1$.

4. Flattening: The resulting pooling output P, which is of size $\frac{(n-k+1)}{s} \times \frac{(m-k+1)}{s}$, is flattened into a vector for input into a fully connected layer:

$$P_{\text{flat}} = \text{Flatten}(P).$$

This pipeline applies a convolutional layer, a detection layer, a pooling layer, and finally prepares the features for fully connected layers in a neural network.

Results

1 Stochastic subgradients for training support vector machines

Given a data set $\{(x_i, y_i)\}_{i=1}^n$ consisting of handwritten digit inputs x_i and their corresponding class labels $y_i \in \{0, 1, ..., 9\}$, our goal is to develop a method that will correctly classify unseen data. We begin by implementing a projected stochastic gradient descent algorithm. The classifier is represented by the matrix $\Theta = [\theta_1, ..., \theta_K] \in R^{d \times K}$, where $d = 64$ represents the size of the input vector x (i.e., the number of pixels in each input image), and $K = 10$ corresponds to the number of classes.

To train the classifier, we aim to minimize the multi-class hinge loss function, defined as:

$$L(\Theta, (x, y)) = \max_{j \neq y} \left[1 + \sum_{i=1}^{d} x_i (\Theta_{ij} - \Theta_{iy}) \right]_+ ,$$

where $[t]_+ = \max(0, t)$ ensures that the loss is non-negative. This function penalizes predictions that fail to maintain a sufficient margin between the true class and the incorrect classes, which is a key concept in support vector machines SVMs.

The goal is thus to minimize the empirical risk function, given by

$$R^{\text{emp}}(\Theta) = \frac{1}{n} \sum_{k=1}^{n} L(\Theta, (x_k, y_k)).$$

By minimizing this function, we wish to improve the classifier's performance on the training dataset, with the goal of generalizing well to unseen data.

1.1 Proving the convexity

In order to use the subgradient descent algorithm, we first need to show that $R^{\text{emp}}(\Theta)$ is a convex function. We know that if $L(\Theta, (x_k, y_k))$ is convex, then the weighted sum over all L will also be convex.

Given an input (x, y), let us write $t_j(\Theta) = 1 + \sum_{i=1}^{d} x_i (\Theta_{ij} - \Theta_{iy})$. Hence,

$$[t_j(\Theta)]_+ = \max(t_j(\Theta), 0).$$

We have that $\nabla_{\theta_{ij}} t_j(\Theta) = x_i$ and $\nabla_{\theta_{iy}} t_j(\Theta) = -x_i$, which are both constants with respect to Θ, making the Hessian 0 everywhere. This implies that $t_j(\Theta)$ is a linear function.

Consider now any $s \in [0, 1]$. For all $x_1, x_2 \in R^d$, we have:

$$\begin{aligned}
[sx_1 + (1-s)x_2]_+ = \max(0, sx_1 + (1-s)x_2) &= \frac{sx_1 + (1-s)x_2 + |sx_1 + (1-s)x_2|}{2} \\
\{\text{triangle inequality}\} &\leq \frac{sx_1 + (1-s)x_2 + |sx_1| + |(1-s)x_2|}{2} \\
&= \frac{sx_1 + |sx_1| + (1-s)x_2 + |(1-s)x_2|}{2} = \{s, (1-s) \geq 0\} \\
&= s\frac{x_1 + |x_1|}{2} + (1-s)\frac{x_2 + |x_2|}{2} = s[x_1]_+ + (1-s)[x_2]_+.
\end{aligned}$$

By (1) we have thus shown that $[x]_+$ is a convex function.

We know that the composition of a convex function with a linear function is convex, thus $[t_j(\Theta)]_+$ is convex. As the maximum of pointwise convex functions preserves convexity, we can thus conclude that $\max_{j \neq y} \left[1 + \sum_{i=1}^d x_i(\Theta_{ij} - \Theta_{iy})\right]_+$ is convex.

Finally, this shows that $R^{\text{emp}}(\Theta) = \frac{1}{n} \sum_{k=1}^n L(\Theta, (x_k, y_k))$ is convex, as it is a weighted sum over a convex function. $\square$

1.2 Finding a Subgradient

Next, we want to find an expression for a subgradient G of $R^{\text{emp}}(\Theta)$ as in (2). That is, we want to find the generalized expression for

$$\nabla_\Theta R^{\text{emp}}(\Theta) = \frac{1}{n} \sum_{k=1}^n \nabla_\Theta L(\Theta, (x_k, y_k)).$$

Consider $j*$ to be the class $j \neq y$ that maximizes the expression $\left[1 + \sum_{i=1}^d x_i(\Theta_{ij} - \Theta_{iy})\right]_+$ for a given (x, y). That is,

$$j* = \arg\max_{j \neq y} \left[1 + \sum_{i=1}^d x_i(\Theta_{ij} - \Theta_{iy})\right]_+.$$

Using this, we can write

$$L(\Theta,(x,y)) = \left[1 + \sum_{i=1}^{d} x_i(\Theta_{ij*} - \Theta_{iy})\right]_+ = \left[1 + \theta_{j*}^T x - \theta_y^T x\right]_+ . \tag{4}$$

The gradient of L can be written as

$$\nabla_\Theta L(\Theta,(x,y)) = \left(\nabla_{\theta_1} L(\Theta,(x,y)), \quad ... \quad , \nabla_{\theta_K} L(\Theta,(x,y))\right), \tag{5}$$

where by (4):

$$\nabla_{\theta_{j*}} L(\Theta,(x,y)) = \begin{cases} x, & \text{if } 1 + \theta_{j*}^T x - \theta_y^T x > 0 \\ 0, & \text{if } \theta_{j*}^T x - \theta_y^T x < 0 \\ g \in [0,x], & \text{if } \theta_{j*}^T x - \theta_y^T x = 0 \end{cases}$$

$$\nabla_{\theta_y} L(\Theta,(x,y)) = \begin{cases} -x, & \text{if } 1 + \theta_{j*}^T x - \theta_y^T x > 0 \\ 0, & \text{if } \theta_{j*}^T x - \theta_y^T x < 0 \\ g \in [-x,0], & \text{if } \theta_{j*}^T x - \theta_y^T x = 0 \end{cases} \tag{6}$$

$$\nabla_{\theta_{j\neq j*,y}} L(\Theta,(x,y)) = 0 \qquad .$$

In the cases where $\theta_{j*}^T x - \theta_y^T x = 0$, we can choose any subgradient g on the interval $[0,x]$ for $\nabla_{\theta_{j*}} L(\Theta,(x,y))$ and any subgradient g on the interval $[-x,0]$ for $\nabla_{\theta_y} L(\Theta,(x,y))$. In our implementation we chose $g = 0$ in both cases.

1.3 Implementing the Projected Subgradient Method

We are now ready to use (6) to implement the projected subgradient method given by (2). We wish to project Θ onto the ball

$$B_r = \left\{ \Theta \in \mathbb{R}^{dk} : \sum_{i,j} \Theta_{ij}^2 \leq r^2 \right\}.$$

We let the training set size, n_{train}, increase, while keeping the test set size to $n_{test} = 100$. The error rate ϵ_k is proportional to $k^{-1/2}$, where k is the iteration. The performance of the model is then assessed by looking at the training error and the test error. This is plotted in figure 1.

As wee can see, the test error dramatically decreases as the training set grows between $n_{train} = 20$ and $n_{train} = 500$, after which it stabilizes somewhat. At $n_{train} = 1500$ and at $n_{train} = 1697$, the test error is 0.03. As for the training error, we see a slight increase as the size of the training set increases. For a training set size of $n_{train} = 20$, the training error is 0.0, but for $n_{train} = 1500$ the training error is 0.0171.

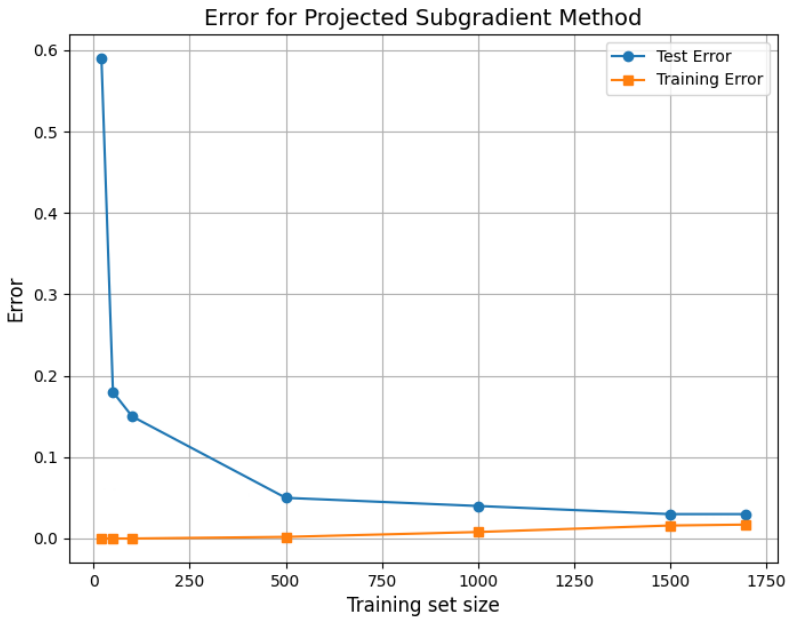

Figure 1: Training Error and Test Error for the Projected Subgradient method, for difference training set sizes, with learning rate $k^{-1/2}$.

Next, we look at the training error and test error as we change the learning rate $\epsilon_k \propto k^{-\alpha}$. We use the full training set with $n_{train} = 1697$, and let α vary between 0 and 1. The results are plotted in figure 2.

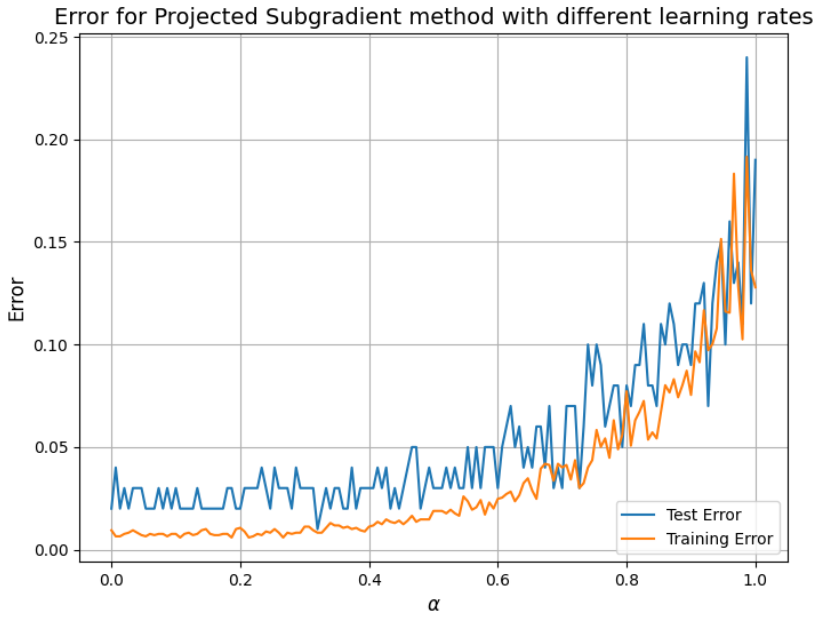

Figure 2: Training Error and Test Error for the Projected Subgradient method, for 150 different learning rates $k^{-\alpha}$ and training set size $n = 1697$.

As we can see in figure 2, the lowest test error obtained is 0.01. This is found at $\alpha = 0.32$. For α-values below 0.32, we find training errors varying between 0.02 and 0.03. For larger α-values, the test error increases more, reaching a maximum of 0.24 for $\alpha = 0.98$. The training error generally also increases as α increases, reaching a maximum of 0.18 at $\alpha = 0.98$. At $\alpha = 0.32$, the training error is 0.008.

The instability of the error for α close to 1 suggests that when α is too large, the learning rate decreases too quickly, preventing the method from accurately approaching the true solution. Conversely, excessively large learning rates (small α) can cause the method to overshoot, thereby missing the optimal classifier. These results thus show the importance of selecting a balanced learning rate to ensure accurate model performance.

2 Gaussian process classification

As in the previous problem we are given training data $\{(x_i, y_i)\}_{i=1}^n$ and ought to classify a set of handwritten digits between 0 and 9, hence the target is a categorical label in $\{0, 1, \ldots, 9\}$. Each digit is stored in a 8×8 pixel black and white image which is represented as a data point $x \in \mathbb{R}^{64}$. The loss function used in this problem is the error rate $\frac{1}{n} \sum_i^n \mathbf{I}\{y_i \neq \hat{y}_i\}$, where $\mathbf{I}$ is the indicator and $\hat{y}_i$ is the predicted class.

2.1 Classification performance of the GP classifier

To investigate the performance of the implemented GP classifier for different kernels the error rate was used as evaluation. The classification is then done by finding the target maximising the predictive posterior $\hat{y} = \arg\max_{c \in \{0,\ldots,9\}} p(y = c \mid x)$. To visualise the error rate we decided to plot the learning curve of the classifier over the relative size of the training data to all data that was used. The comparison was done by comparing an RBF-kernel with a dot product and a Matérn kernel as well as with two different combinations of these. A list of these covariance functions can be found in the appendix in (7).

The error rate is plotted against the relative training size.

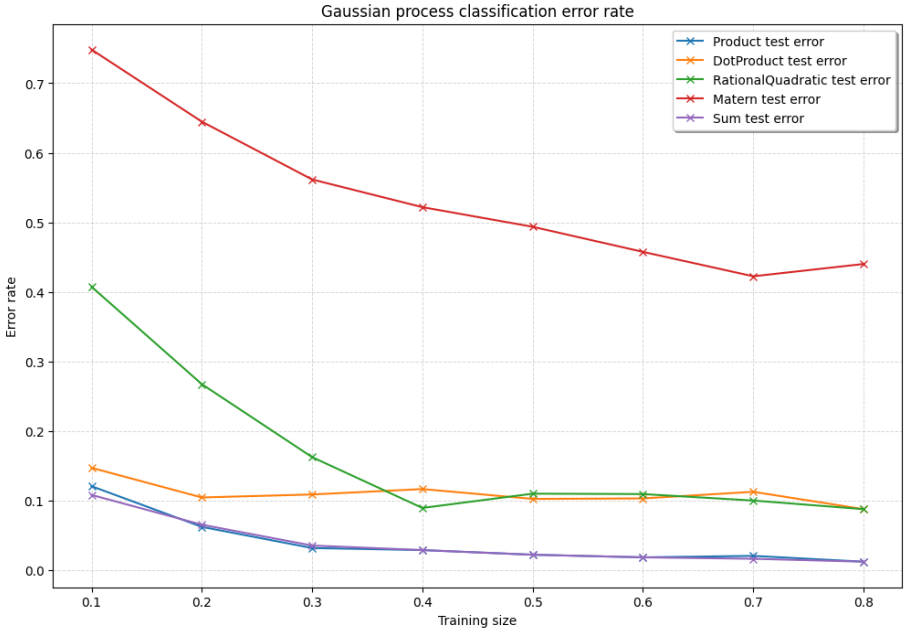

Figure 3: Test error rate of the GP classifier using RBF, dot product and Matérn as well as a combination of them as covariance functions. Training error for all kernels were almost 0, not providing much to the plot.

As seen in figure 3 the covariance function providing the minimal test error rate is the "sum" and the "product", which are the mixed kernel and the half RBF kernel, respectively, as can be seen in (7). The error rate was as low as 0.008 for these, when trained on 80% of the used data.

2.2 Classification performance of multi-class classifier GP regression

To implement the multi-class classifier, the K classes were one-hot encoded and GP regression was then applied to predict the numerical values of the K-dimensional encoded response variable. When the regression has been implemented, the classification was made by choosing the class maximising the regression response variable at each point. As in the classifier case, the performance was measured in error rate.

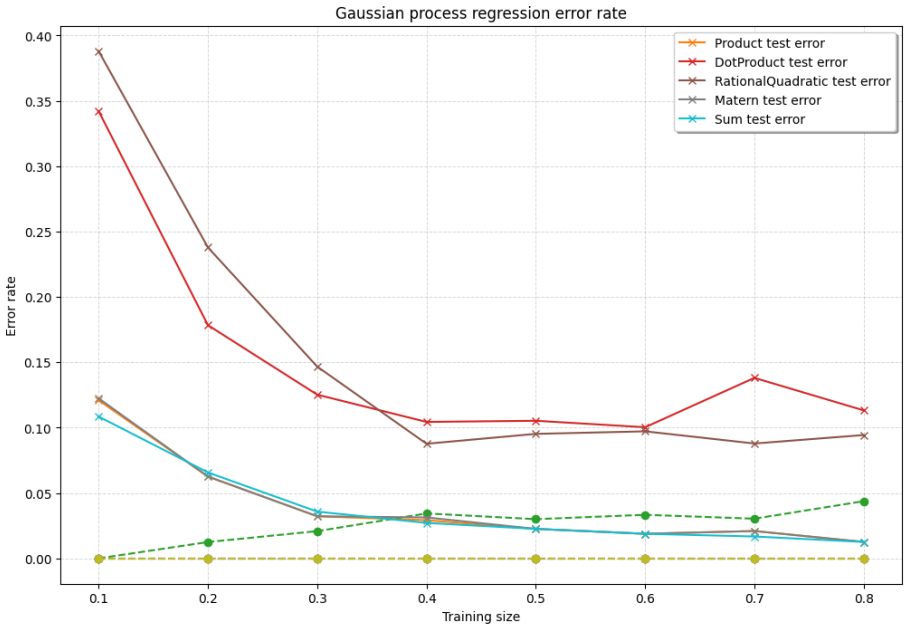

Figure 4: Training and test error rate of the multi-class GP classifier using RBF, dot product and Matern as covariance functions.

In figure 4 it can be seen that the RBF and mixed kernel are the ones minimising the error rate. Moreover, the accuracy of the Matérn kernel has decreased for the multi-class regression, while the other kernels has not changed much from the ordinary GP classification. The error rate was about 0.008 when trained on 80% of the used data.

3 Convolutional neural network (CNN) classification

3.1 Classification performance for out-of-the-box CNN

To classify the images in the digits dataset, a Convolutional Neural Network (CNN) was implemented using TensorFlow. The model used the following configuration:

- **Number of filters (kernels):** 16
- **Filter size:** 3×3
- **Pooling size:** 2×2
- **Fully connected (dense) layer neurons:** 64

The CNN was trained for 20 epochs with a batch size of 32. Below are the plots for the training and validation accuracy and loss.

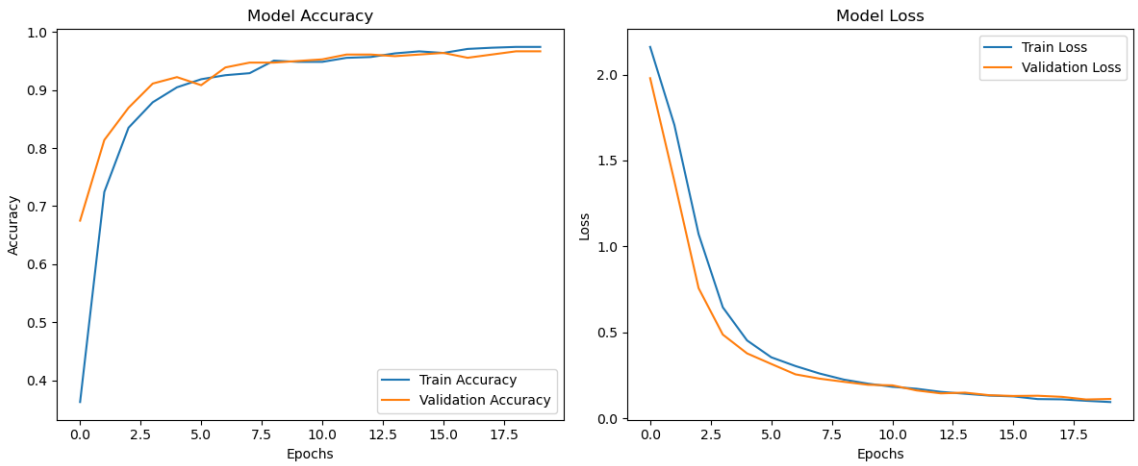

Figure 5: Training and validation accuracy (left) and loss (right) over 20 epochs.

We can see that the training accuracy converges quickly, reaching above 90% within the first 5 epochs, and stabilises around 99% by epoch 20. The validation accuracy follows a similar trend, stabilising at approximately 98%, indicating that the model generalises well to unseen data.

The training loss decreases steadily, reaching near-zero values by the end of training. The validation loss decreases rapidly in the initial epochs, stabilizing after epoch 5, indicating that the model is not overfitting significantly.

3.2 Investigate the performance of the CNN with various parameters

To evaluate the performance of the CNN in terms of training data size and layer design, multiple configurations were tested by varying:

- **Fully connected (dense) layer neurons:** 16, 32, 64, 128
- **Number of filters (kernels):** 16, 32, 64, 128
- **Filter size:** 3×3, 5×5, 7×7
- **Pooling size:** 2×2, 3×3

The results were analysed in terms of the number of parameters and the corresponding test error. The configurations that produced the best trade-off between performance and parameter count are highlighted in Figure 6.

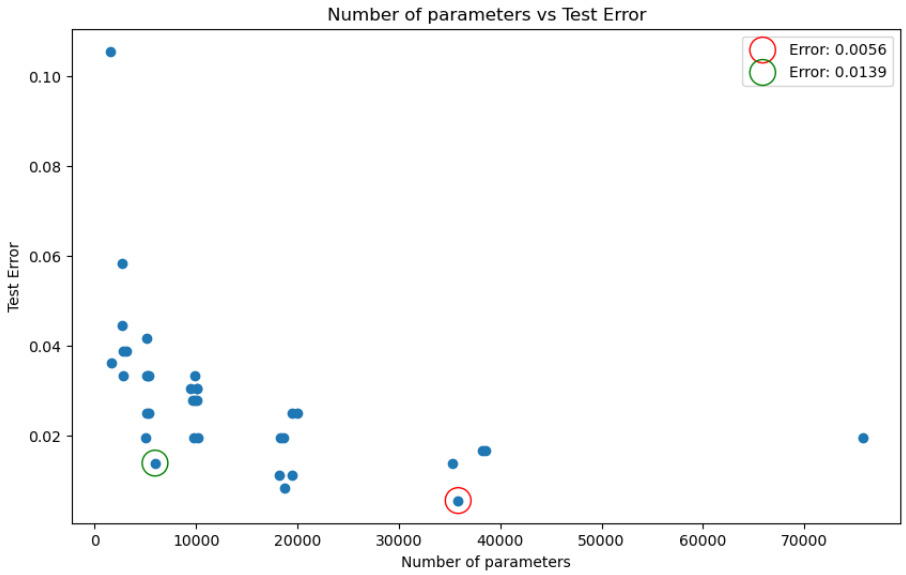

Figure 6: Test error vs. number of parameters for different CNN configurations. The best configuration (red circle) and the chosen configuration (green circle)

The best configuration (red circle) had the parameters (128, 64, (5, 5), (2, 2), 35850), in the order of the bullet points above with the addition of number of parameters. It seems to be minimising the test error to 0.0056 with relatively high parameters, while the chosen configuration (green circle) (16, 64, (5, 5), (2, 2), 5946) achieves a balance between parameter count and performance with the test error being 0.0139, with the mere difference of 0.0083. We deem this to be an insignificant decline in performance given the fact that

we have 6x less the number of parameters. We also note that the improvement in test error seems to decrease exponentially with the number of parameters.

Summary

In summary, we have looked at three different approaches for classifying low-resolution hand-written digits, all of which have shown to be successful.

Using a convex empirical risk function, the **Support Vector Machine** achieved a classification accuracy of 99% on unseen data. This performance was based on a full training set of size 1697 and a learning rate $\propto k^{-\alpha}$ with $\alpha = 0.32$. For α values near 0.32, the algorithm also obtained high precision, correctly classifying 97–98% of unseen data. However, for $\alpha > 0.48$, the results became more unstable.

When implementing a **Gaussian Process** classifier, we reached a maximum accuracy of 99.2%. This was obtained using both the mixed kernel ("sum") and the half RBF kernel ("product"), and a training set of size $n_{train} = 1437$.

As for the multi-class GP classifier with one-hot encoded classes, an accuracy of 99.2% was also obtained using the mixed kernel and the RBF kernel, with $n_{train} = 1437$. These results suggest that there was no measurable difference between the predictive power of the two GP methods.

Finally, using a **Convolutional Neural Network**, we obtained a maximum test accuracy of 99.44%, after tuning the parameters. However, in the balance between high accuracy and a small number of parameters, we found that the optimal model has an accuracy of 98.61%. This CNN used 16 neurons in the NN and 64 kernels, a filter size of 5×5, a pooling size of 2×2, amounting to a total of 5946 fully connected layer neurons.

In conclusion, the prediction accuracy on unseen data was approximately 99% for all three classifiers, which shows that all three are suitable methods for classifying low-resolution handwritten digits. However, if a dataset with higher resolution images was used, the results may differ. Neural networks, particularly CNNs, are known for their ability to capture complex structures, which suggests that a CNN could perform well in such scenarios, although it may be susceptible to overfitting. For the SVM classifier, performance could potentially be enhanced by exploring alternative generalized gradients rather than consistently using zero as the subgradient for the undecided cases. Finally, for the Gaussian process (GP) classifier, further investigation of different kernels could potentially provide better model accuracy if dealing with more complex datasets.

Appendix

The kernels used in the Gaussian process classification and multi class regression were the following:

$$
\begin{aligned}
k(x, x') &= k_{\text{Matérn}}(\|x - x'\|), \text{where} \\
k_{\text{Matérn}}(r) &= \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} r}{\ell} \right)^{\nu} K_{\nu} \left(\frac{\sqrt{2\nu} r}{\ell} \right), \ \nu = 2.5, \ \ell = 1, \\
\frac{1}{2} k_{\text{rbf}}(x, x') &= \frac{1}{2} \exp\left(-\frac{\|x - x'\|^2}{2\ell^2} \right), \ \ell = 1 \\
k_{\text{dot}}(x, x') &= x^{\top} x' + \sigma^2, \ \sigma^2 = 1, \\
k_{\text{mix}}(x, x') &= k_{\text{rbf}}(x, x'; \ell = 9) + k_{\text{dot}}(x, x'; \sigma^2 = 3.5) \cdot k_{\text{Matérn}}(\|x - x'\|; \ell = 2).
\end{aligned}
\tag{7}
$$

Listing 1: Code for SVM Classifier

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.utils import shuffle
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF
from sklearn.gaussian_process.kernels import DotProduct

# import some data to play with
digits = datasets.load_digits()

#choose a seed
seed = 1010

np.random.seed(seed)
# Load data into train set and test set
digits = datasets.load_digits()
X = digits.data
y = np.array(digits.target, dtype = int)
X,y = shuffle(X,y)
N,d = X.shape



def svmsubgradient(Theta, x, y):
#  Returns a subgradient of the objective empirical hinge loss
# The inputs are Theta, of size n-by-K, where K is the number of classes,
# x of size n, and y an integer in {0, 1, ..., 9}.
    G = np.zeros(Theta.shape) # shape is (n,K)

    #SUBGRADIENT CALCULATIONS
    K = Theta.shape[1] # number of classes = 10
    t_values = [] # save the values of the loss expression
    for j in range(K):
        if j != y:
            t_j = 1 + np.dot(x, Theta[:, j] - Theta[:, y])
            t_values.append((j, t_j))

    j_star, _ = max(t_values, key=lambda item: item[1]) # find the j that
        maximizes this
    hinge_loss = 1 + np.dot(x, Theta[:, j_star] - Theta[:, y]) # if this is
         > 0, set gradients accordingly

    if hinge_loss > 0:
        G[:, j_star] = x  # update the gradient for class j*
        G[:, y] = -x   # update the gradient for class y

    # since the subgradient is 0 in all other cases, we don't have to
        update anything if hinge loss <= 0
```

```
    return(G)

def sgd(Xtrain, ytrain, maxiter = 10, init_stepsize = 1.0, l2_radius =
    10000, learning_rate = 1/2):
# Performs maxiter iterations of projected stochastic gradient descent
# on the data contained in the matrix Xtrain, of size n-by-d, where n
# is the sample size and d is the dimension, and the label vector
# ytrain of integers in {0, 1, ..., 9}. Returns two d-by-10
# classification matrices Theta and mean_Theta, where the first is the
    final
# point of SGD and the second is the mean of all the iterates of SGD.
# Each iteration consists of choosing a random index from n and the
# associated data point in X, taking a subgradient step for the
# multiclass SVM objective, and projecting onto the Euclidean ball
# The stepsize is init_stepsize / sqrt(iteration).
    K = 10
    NN, dd = Xtrain.shape
    #print(NN)
    Theta = np.zeros(dd*K)
    Theta.shape = dd,K
    mean_Theta = np.zeros(dd*K)
    mean_Theta.shape = dd,K

    for i in range(maxiter):
        stepsize = init_stepsize / ((i+1)**learning_rate)
        n = np.random.randint(1, NN)
        x_n = Xtrain[n,:]
        y_n = ytrain[n]
        gn = svmsubgradient(Theta, x_n, y_n)
        Theta = Theta - stepsize*gn

        # projection onto the ball:
        theta_sum = np.sum(np.square(Theta))
        if theta_sum > l2_radius**2:
            Theta = np.sqrt(l2_radius**2 / theta_sum) * Theta

        mean_Theta += Theta/maxiter

    return Theta, mean_Theta

def Classify(Xdata, Theta):
# Takes in an N-by-d data matrix Adata, where d is the dimension and N
# is the sample size, and a classifier X, which is of size d-by-K,
# where K is the number of classes.
# Returns a vector of length N consisting of the predicted digits in
# the classes.
    scores = np.matmul(Xdata, Theta)
    inds = np.argmax(scores, axis = 1)
    return(inds)
```

```
Ntest = int(100)
l2_radius = 40.0
maxiter = 40000

def plot_different_training_size():
    training_set_size = [20, 50, 100, 500, 1000,1500, 1697]
    test_error_rates = np.zeros(len(training_set_size))
    train_error_rates = np.zeros(len(training_set_size))
    Xtest = X[N-100:N,:]
    ytest = y[N-100:N]


    learning_rate = 1/2
    for i in range(len(training_set_size)):
        Ntrain = training_set_size[i]
        Xtrain = X[0:Ntrain,:]
        ytrain = y[0:Ntrain]
        M_raw = np.sqrt(np.mean(np.sum(np.square(Xtrain))))
        init_stepsize = l2_radius/M_raw
        Theta, mean_Theta = sgd(Xtrain, ytrain, maxiter, init_stepsize,
            l2_radius, learning_rate)
        test_error_rate = np.sum(np.not_equal(Classify(Xtest, mean_Theta),
            ytest)/Ntest)
        train_error_rate = np.sum(np.not_equal(Classify(Xtrain, mean_Theta)
            ,ytrain)/Ntrain)
        test_error_rates[i] = test_error_rate
        train_error_rates[i] = train_error_rate

        print("train␣size:␣", Ntrain, "test␣error:␣", test_error_rate)

    print("training␣error:␣", train_error_rates)
    plt.figure(figsize=(8, 6))
    plt.plot(training_set_size,test_error_rates, label='Test␣Error',
        linestyle='-', marker='o')
    plt.plot(training_set_size,train_error_rates, label='Training␣Error',
        linestyle='-', marker='s')
    plt.xlabel('Training␣set␣size', fontsize=12)
    plt.ylabel('Error', fontsize=12)
    plt.title('Error␣for␣Projected␣Subgradient␣Method', fontsize=14)
    plt.legend(loc='upper␣right', fontsize=10)
    plt.grid(True)
    plt.show()



def plot_different_learning_rates():
    nr_rates = 151
    alpha = np.linspace(0, 1, nr_rates).tolist()
    test_error_rates = np.zeros(nr_rates)
    train_error_rates = np.zeros(nr_rates)
```

```
    Ntrain = 1697
    Xtrain = X[0:Ntrain,:]
    ytrain = y[0:Ntrain]
    Xtest = X[N-100:N,:]
    ytest = y[N-100:N]
    M_raw = np.sqrt(np.mean(np.sum(np.square(Xtrain))))
    init_stepsize = l2_radius/M_raw

    for i in range(nr_rates):
        rate = alpha[i]
        Theta, mean_Theta = sgd(Xtrain, ytrain, maxiter, init_stepsize,
            l2_radius, rate)
        test_error_rate = np.sum(np.not_equal(Classify(Xtest, mean_Theta),
            ytest)/Ntest)
        train_error_rate = np.sum(np.not_equal(Classify(Xtrain, mean_Theta)
            ,ytrain)/Ntrain)
        test_error_rates[i] = test_error_rate
        train_error_rates[i] = train_error_rate

        print("rate␣",rate, ",␣test␣error", test_error_rate, ",␣train␣error
            :␣",train_error_rate)

    plt.figure(figsize=(8, 6))
    plt.plot(alpha,test_error_rates, label='Test␣Error', linestyle='-')#,
        marker='o')
    plt.plot(alpha,train_error_rates, label='Training␣Error', linestyle='-'
        )#, marker='s')
    plt.xlabel(r'$\alpha$', fontsize=12)
    plt.ylabel('Error', fontsize=12)
    plt.title('Error␣for␣Projected␣Subgradient␣method␣with␣different␣
        learning␣rates', fontsize=14)
    plt.legend(loc='lower␣right', fontsize=10)
    plt.grid(True)
    plt.show()



"plot␣training␣size␣accuracies"
plot_different_training_size()

"plot␣learning␣rates"
plot_different_learning_rates()
```

Listing 2: Code for GP Classifier

```
"""
=========================================================
Gaussian process classification (GPC)
=========================================================

"""

import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.utils import shuffle
from sklearn.gaussian_process import GaussianProcessClassifier,
    GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, DotProduct, Matern,
    RationalQuadratic, ExpSineSquared
from sklearn.preprocessing import LabelBinarizer
# from sklearn.model_selection import train_test_split # To split data into
     random train and test sets, for more robust comparison

#GaussianProcessClassifier performs hyperparameter estimation, this means
    that the the value specified above may not be the final hyperparameters
#If you dont want it to do hyperparameter optimization set optimizer = None
     like this GaussianProcessClassifier(kernel=kernel,optimizer = None).fit
    (Xtrain, ytrain)
#You can check the final hyperparameters with gpc_rbf.kernel_.get_params()
    ['kernels']

def gp_classifier(kernels, training_sizes, N, X, y):
    results = {kernel.__class__.__name__: {"Train␣error␣rate": [], "Test␣
        error␣rate": []} for kernel in kernels} # Storing the result of each
         kernel for different training sizes
    for Ntrain in training_sizes:
        Ntest = N - Ntrain
        Xtrain = X[0:Ntrain,:]
        ytrain = y[0:Ntrain]
        Xtest = X[Ntrain+1:N,:]
        ytest = y[Ntrain+1:N]

    ### looping through kernels
        for kernel in kernels:
            kernel_name = kernel.__class__.__name__
            gpc_rbf = GaussianProcessClassifier(kernel=kernel).fit(Xtrain,
                ytrain)
            yp_train = gpc_rbf.predict(Xtrain)
            train_error_rate = np.mean(np.not_equal(yp_train,ytrain))
            yp_test = gpc_rbf.predict(Xtest)
            test_error_rate = np.mean(np.not_equal(yp_test,ytest))

            results[kernel_name]["Train␣error␣rate"].append(
```

```
                train_error_rate)
            results[kernel_name]["Test error rate"].append(test_error_rate)
    return results

def gp_regression(kernels, training_sizes, N, X, y):
    results = {kernel.__class__.__name__: {"Train error rate": [], "Test
        error rate": []} for kernel in kernels} # Storing the result of each
         kernel for different training sizes
    lb = LabelBinarizer()
    y_one_hot = lb.fit_transform(y)
    for Ntrain in training_sizes:
        Ntest = N - Ntrain
        Xtrain = X[0:Ntrain,:]
        ytrain = y[0:Ntrain]
        Xtest = X[Ntrain+1:N,:]
        ytest = y[Ntrain+1:N]
        ytrain_one_hot = lb.transform(ytrain)
        ytest_one_hot = lb.transform(ytest)

    ### looping through kernels
        for kernel in kernels:
            kernel_name = kernel.__class__.__name__
            # GP regression for each one hot encoded categopry of y
            gp_regs = [GaussianProcessRegressor(kernel=kernel).fit(Xtrain,
                ytrain_one_hot[:, i]) for i in range(ytrain_one_hot.shape
                [1])]
            # Predicting for each regression of which the highest score is
                to be used
            yp_train_scores = np.array([gp_reg.predict(Xtrain) for gp_reg
                in gp_regs])
            # Finding the class with the largest predicted value
            yp_train = np.argmax(yp_train_scores, axis=0)
            train_error_rate = np.mean(np.not_equal(yp_train,ytrain))

            yp_test_scores = [gp_reg.predict(Xtest) for gp_reg in gp_regs]
            yp_test = np.argmax(yp_test_scores, axis=0)
            test_error_rate = np.mean(np.not_equal(yp_test,ytest))

            results[kernel_name]["Train error rate"].append(
                train_error_rate)
            results[kernel_name]["Test error rate"].append(test_error_rate)
    return results

def plot_error_rate(training_sizes, results, save_path, title):
    plt.figure(figsize=(12, 8))
    for kernel_name in results:
        # print(f'{kernel_name} Test error rate')
        # print(results[kernel_name]["Test error rate"])
        plt.plot(training_sizes , results[kernel_name]["Train error rate"],
             linestyle='--', marker='o')
```

```
        plt.plot(training_sizes, results[kernel_name]["Test error rate"],
            label=f"{kernel_name} test error", linestyle='-', marker='x')
    plt.grid(alpha=0.5, linestyle='--', linewidth=0.7)
    plt.xlabel("Training size")
    plt.ylabel("Error rate")
    plt.title(title)
    plt.legend(loc="upper right", fontsize=10, frameon=True, shadow=True)
    plt.savefig(save_path, bbox_inches='tight')
    plt.show()

### Main function to run the code
def main():
    #choose a seed
    seed = 101
    np.random.seed(seed)

    # import some data to play with
    digits = datasets.load_digits()

    X = digits.data
    y = np.array(digits.target, dtype = int)
    X,y = shuffle(X,y)
    N,d = X.shape

    # N = int(600) # Trying different n, but for consistency i lab we keep
        it as for the other implementations

    kernel1 = 0.5*RBF([5.0])
    kernel2 = RBF([9.0]) + DotProduct(3.5)*Matern(length_scale=2)
    ### Defining more kernels
    kernels = [kernel1,
                DotProduct(1.0),
                RationalQuadratic(length_scale=1.0),
                Matern(length_scale=1.0, nu=2.5),
                kernel2
            ]

    training_sizes = [int(m*N/10) for m in range(1,9)]
    training_shares = np.array(training_sizes)/N

    results_gpc = gp_classifier(kernels, training_sizes, N, X, y)
    results_gp_reg = gp_regression(kernels, training_sizes, N, X, y)

    plot_error_rate(training_shares, results_gpc, save_path="
        GP_classification_error_rate.png", title="Gaussian process
        classification error rate")
    plot_error_rate(training_shares, results_gp_reg, save_path="
        GP_regression_error_rate.png", title="Gaussian process regression
        error rate")
```

```
if __name__ == "__main__":
    main()
```

Listing 3: Code for CNN Classifier

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense


""" -------------- Part 1 --------------"""

# Function to initialise the model
def initialise_model(d=64, filter_size=(3, 3), pool_size=(2, 2),
    num_of_filter=16):
    model = Sequential([
        Conv2D(num_of_filter, filter_size, activation='relu', input_shape
            =(8, 8, 1)),
        MaxPooling2D(pool_size=pool_size),
        Flatten(),
        Dense(d, activation='relu'),
        Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy',
        metrics=['accuracy'])


    # Print the model summary
    model.summary()
    return model


digits = datasets.load_digits()

# Extract featires and labels
X = digits.images
y = digits.target

# Preprocess the data, normalise the pixel values
X = X.reshape(X.shape[0], 8, 8, 1)  # (samples, 8x8 image, 1 channel) is
    CNN input format
X = X / X.max() # X_max is 16

# One-hot encode the labels
y = to_categorical(y, num_classes=10)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
```

```
print("Shape of X_train:", X_train.shape)
print("Shape of y_train:", y_train.shape)


model = initialise_model()

# Train the model on training data
history = model.fit(X_train, y_train, validation_data=(X_test, y_test),
    epochs=20, batch_size=32)

# Evaluate model performance on the test data
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Plot accuracy and loss during training
plt.figure(figsize=(12, 5))

# Plot accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')

plt.ylabel('Accuracy')
plt.legend()

# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

""" -------------- Part 2 --------------"""
dense_units = [16, 32, 64, 128]
filter_units = [16, 32, 64]
filter_sizes = [(3, 3), (4, 4), (5, 5)]
pool_sizes = [(2, 2)]

results = {}

for dense_unit in dense_units:
```

```
    for filter_unit in filter_units:
        for filter_size in filter_sizes:
            for pool_size in pool_sizes:
                feature_map_size = 8 - filter_size[0] + 1

                # Skip configurations where pooling size exceeds the
                    feature map size
                if feature_map_size < pool_size[0]:
                    continue
                print(f"Training␣model␣with␣dense␣unit:␣{dense_unit},␣
                    filter␣unit:␣{filter_unit},␣filter␣size:␣{filter_size},␣
                    pool␣size:␣{pool_size}")
                model = initialise_model(d=dense_unit, filter_size=
                    filter_size, pool_size=pool_size, num_of_filter=
                    filter_unit)
                history = model.fit(X_train, y_train, validation_data=(
                    X_test, y_test), epochs=20, batch_size=32, verbose=0)
                test_loss, test_accuracy = model.evaluate(X_test, y_test,
                    verbose=0)
                num_params = model.count_params()
                results[(dense_unit, filter_unit, filter_size, pool_size,
                    num_params)] = test_accuracy

best_config = max(results, key=results.get)

print(f"Best␣configuration:␣{best_config}")

# chosen_config = (16, 64, (5, 5), (2, 2), 5946) # for the hand-in
    assignment
chosen_config = (64, 64, (5, 5), (3, 3), 6474)

# Print num_params against accuracy
plt.figure(figsize=(10, 6))
plt.scatter([key[-1] for key in results.keys()], [1 - value for value in
    results.values()])
# Circle in the point with index 8
plt.scatter(best_config[-1], 1 - results[best_config], facecolors='none', s
    =300, edgecolor='red', label=f'Error:␣{1-results[best_config]:.4f}')
plt.scatter(chosen_config[-1], 1 - results[chosen_config], facecolors='none
    ', s=300, edgecolor='green', label=f'Error:␣{1-results[chosen_config]:.4
    f}')
plt.xlabel("Number␣of␣parameters")
plt.ylabel("Test␣Error")
plt.title("Number␣of␣parameters␣vs␣Test␣Error")
plt.legend()
plt.show()

print(f"Best␣configuration:␣{best_config}")
print(f"Chosen␣configuration:␣{chosen_config}")
```

```
# To locate config in the plot and mark it in the plot above (i.e. to
    decide chosen_config)
i=0
cur_best_accuracy = (0,0,0) # (index, accuracy, config
for key, value in results.items():

    if key[-1] > 5000 and key[-1] < 9000:
        if value > cur_best_accuracy[1]:
            cur_best_accuracy = (i, value, key)
    i+=1

print(cur_best_accuracy)
```